Università di Pisa

University of Pisa

Department of Information Engineering

Master's Degree in Computer Engineering

Advanced Computer Networking Course

Network Automation Project

Authors:

Valerio Triolo
Luca Arduini

Academic Year: 2025/2026

Contents

Introduction **3**
0.1 Project Context . . . 3
0.2 Objectives . . . 3

1 Network Architecture **4**

2 Task 1: Configuration templates **7**
2.1 Data model . . . 7
2.2 Configuration Generation Script . . . 8
2.2.1 Initialization Phase . . . 8
2.2.2 Helper Functions . . . 8
2.2.3 Per-Node Configuration Logic . . . 9
2.2.4 Template Rendering . . . 9
2.2.5 Automation Benefits . . . 10
2.3 Output . . . 10

3 Task 2: Network emulation with containerlab **11**
3.1 Components . . . 11
3.2 Automation . . . 11
3.2.1 Data Validation Phase . . . 12
3.2.2 Template-Based Topology Generation . . . 14

4 Task 3: Traffic engineering **15**
4.1 System Architecture and Traffic Matrix Generation . . . 15
4.1.1 Traffic Matrix Structure . . . 15
4.2 Optimization Strategy . . . 16
4.2.1 Handling Null Traffic and Default Routing . . . 17
4.2.2 Actuation Mechanism: BGP Attributes . . . 17
4.3 Ingress Traffic Engineering: PE Selection Optimization . . . 17
4.3.1 Problem Definition . . . 17
4.3.2 Initial Formulation . . . 18
4.3.3 Mixed-Integer Linear Programming (MILP) Model . . . 18
4.3.4 Implementation and Resolution . . . 19
4.3.5 From Optimization to Configuration: BGP MED . . . 19
4.4 Egress Traffic Engineering: Gateway Selection Optimization . . . 20
4.4.1 Problem Definition . . . 20
4.4.2 Mathematical Model: Minimax Saturation . . . 20

4.4.3 From Optimization to Configuration: BGP Local Preference . 21
4.5 Automated Configuration Deployment 22
4.5.1 Interaction with the Emulated Control Plane 22
4.5.2 Applying BGP Attributes via Route Maps 22

5 Deployment and Automation Pipelines 24
5.1 Host Machine Setup: `install.py` 24
5.2 Orchestration and Deployment: `orchestrator.py` 24
5.2.1 Artifact Generation 24
5.2.2 Synchronization with the VM 25
5.3 Infrastructure Bootstrapping: `bootstrap.sh` 25
5.3.1 Connectivity and Environment Checks 25
5.3.2 Building the Manager Node 26
5.3.3 Topology Instantiation 26
5.3.4 Environment Cleanup: `teardown.sh` 26
5.4 Traffic Engineering Execution: `manager.py` 26
5.5 Manager Execution Output 27

6 Testing and Results 30
6.1 Data Plane Connectivity and Routing 30
6.1.1 Bidirectional Reachability Analysis 30
6.1.2 Host Routing Verification 31
6.2 Traffic Engineering Validation 31
6.2.1 Methodology 31
6.2.2 Link-Level IP Resolution 32
6.2.3 Validation Logic 32
6.2.4 Results Interpretation 32

7 Future Work and Scalability Analysis 33
7.1 Topology Generalization and Testing 33
7.2 Algorithmic Scalability 33

8 AI Usage Disclaimer 35
8.1 Code Readability and Presentation 35
8.2 Documentation Support 35

Introduction

This report presents the design, implementation and validation of an automated Network Traffic Engineering system applied to a Transit Autonomous System.

The code is available on GitHub at: github.com/LucaArduini/ACN_bgp-automation

0.1 Project Context

This project focuses on the design, emulation, and automation of a complex network infrastructure, specifically targeting the operations within Autonomous System AS65020. As modern networks grow in complexity, manual configuration and static routing policies become inefficient and error-prone. Consequently, this work aims to implement a fully automated pipeline that integrates network automation principles with dynamic traffic engineering.

0.2 Objectives

The main objectives of this work are:

1. **Configuration Management:** Utilizing Jinja2 templating to generate consistent, reusable and parameterized configurations for network devices. This ensures scalability and minimizes human error in defining router IDs, interface IPs, and BGP neighbor relationships.

2. **Network Emulation:** Deploying a realistic network topology using Containerlab and FRRouting (FRR). This lightweight, container-based approach allows for the rapid instantiation of the network environment to verify routing protocols and topology characteristics before production deployment.

3. **Dynamic Traffic Engineering:** The core of the project involves developing an automation system hosted on a manager node within AS65020. This system is designed to ingest periodic traffic prediction matrices, analyze optimal distribution strategies, and automatically manipulate BGP attributes (specifically `MED` and `LOCAL_PREF`). This closed-loop system allows the network to adapt to predicted traffic loads by steering traffic across upstream links without manual intervention.

Chapter 1

Network Architecture

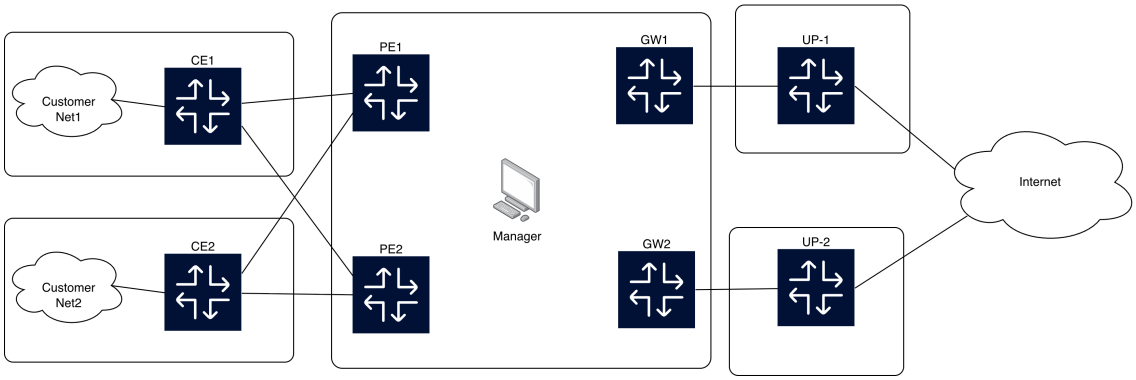

Figure 1.1: The original proposed architecture

The network to automate is represented in Figure 1.1. The customer's routers `CE1` and `CE2` are both linked with `PE1` and `PE2` for redundancy and reliability. For sake of simplicity, the provider edge routers are in the same network as the gateways, linked with a lan bridge.

The customer networks are implemented using a node (N1/N2) directly connected to CE1/CE2 in the same autonomous system.

In this particular case, N1 and N2 can communicate via the transit network.

The internet is represented by three routers R1, R2 and R3 connected with both UP-1 and UP-2.

The new network architecture can be seen better in the new Figure 1.2 and in Table 1.1 .

Table 1.1: Nodes, Interfaces and IP Addresses

Node	**Role**	**Interface**	**IP Address**	**Mask**
n1	host	eth1	198.51.100.2	/24
n1	host	node-ip	198.51.100.3	/24
n2	host	eth1	192.0.0.1	/24
n2	host	node-ip	192.0.0.3	/24
ce1	ce	eth3	198.51.100.1	/24
ce1	ce	eth1	10.1.11.1	/30

Node	Role	Interface	IP Address	Mask
ce1	ce	eth2	10.1.12.2	/30
ce2	ce	eth3	192.0.0.2	/24
ce2	ce	eth1	10.2.0.1	/30
ce2	ce	eth2	10.2.11.6	/30
pe1	pe	lo	10.0.0.11	/32
pe1	pe	eth1	10.1.11.2	/30
pe1	pe	eth3	10.2.0.2	/30
pe1	pe	eth4	192.168.12.1	/24
pe2	pe	lo	10.0.0.12	/32
pe2	pe	eth1	10.1.12.1	/30
pe2	pe	eth3	10.2.11.5	/30
pe2	pe	eth4	192.168.12.3	/24
gw1	gw	lo	192.168.0.11	/32
gw1	gw	eth3	192.168.12.2	/24
gw1	gw	eth2	172.30.1.1	/30
gw2	gw	lo	10.3.0.1	/32
gw2	gw	eth3	192.168.12.4	/24
gw2	gw	eth2	172.30.2.1	/30
up1	up	lo	203.0.99.1	/32
up1	up	eth1	172.30.1.2	/30
up1	up	eth2	10.4.1.1	/30
up1	up	eth3	10.3.1.1	/30
up1	up	eth4	10.2.1.1	/30
up2	up	lo	10.3.1.11	/32
up2	up	eth1	172.30.2.2	/30
up2	up	eth2	10.4.2.1	/30
up2	up	eth3	10.3.2.1	/30
up2	up	eth4	10.2.2.1	/30
r1	router	lo	8.8.8.8	/32
r1	router	eth1	10.4.1.2	/30
r1	router	eth2	10.4.2.2	/30
r2	router	lo	7.7.7.7	/32
r2	router	eth1	10.3.1.2	/30
r2	router	eth2	10.3.2.2	/30
r3	router	lo	6.6.6.6	/32
r3	router	eth1	10.2.1.2	/30
r3	router	eth2	10.2.2.2	/30
manager	host	lo	192.168.12.24	/32
manager	host	eth1	192.168.12.5	/24

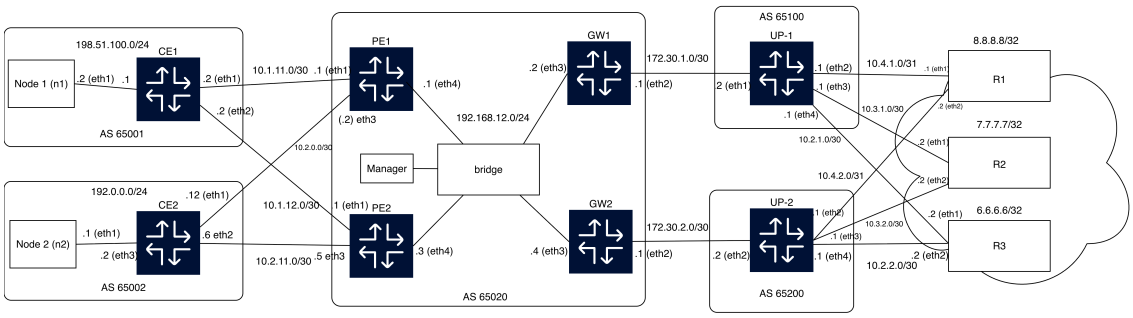

Figure 1.2: New topology

Chapter 2

Task 1: Configuration templates

The primary objective of this task was to automate the generation of configuration files for the network nodes (routers, PEs, CEs, GWs). Instead of manually configuring each device—which is prone to human error and difficult to scale—we implemented a data-driven automation approach, ensuring consistency, scalability, and ease of modification across the entire topology. There is a data model, in form of a file `data.yaml` that is used to define the topology, IP addressing, AS numbers, and relationships between nodes. **Jinja2** is used to create a dynamic configuration skeleton (`config.j2`) compatible with the FRR routers syntax. There is a python script, `build_config.py`, that parses the data model, computes logical relationships (neighbors, routes), and renders the final configurations for each device.

2.1 Data model

The topology is defined in `data.yaml`. Each node is specified inside the attribute `nodes` and for each node it must be specified:

- **Role**: defines the function (router, gateway, host). Hosts are excluded from router configuration generation but are used to calculate gateway requirements.
- **IPv4 Address**: The IPv4 address assigned to that machine. Usually this address is represented by the loopback address (/32).
- **BGP Attributes**: including the AS number and the router id.
- **Interfaces**: a list of physical interfaces with assigned IPv4 addresses and masks.

For specifying the links (connection between nodes) the `links` attribute is used. For each link you need to specify:

- **a**: source device
- **a_port**: source device interface
- **b**: destination device
- **b_port**: destination device interface

2.2 Configuration Generation Script

The script `build_config.py` is responsible for automatically generating FRR-compatible configuration files for all routing devices defined in the topology. Its purpose is to transform the abstract topology description contained in `data.yaml` into concrete per-device configuration files.

2.2.1 Initialization Phase

At startup, the script:

- Defines the working directories (templates, topology data, and output directory).
- Creates the output directory if it does not already exist.
- Loads the Jinja2 template engine and imports the configuration template `config.j2`.
- Reads and parses the topology file `data.yaml`.

The parsed YAML file provides two main data structures:

- `nodes`: all devices in the topology.
- `links`: all physical connections between devices.

To simplify lookups, the script builds a dictionary (`nodes_map`) that maps each node name to its full definition. This enables fast access when computing neighbor relationships.

2.2.2 Helper Functions

Two helper functions are defined to support the configuration logic:

- `get_remote_ip(node_name, port_name)`: retrieves the IPv4 address assigned to a specific interface of a given node.
- `get_network_address(ip, mask)`: computes the network prefix associated with an IP address and subnet mask using Python's `ipaddress` module.

These functions allow the script to dynamically derive routing and BGP network statements without hardcoding prefixes.

2.2.3 Per-Node Configuration Logic

The script iterates over all nodes and generates configuration files only for routing devices. Hosts are skipped, since they do not require FRR routing configuration.

For each routing node, the script performs the following steps:

1. **Extract BGP Information**
   If the node has BGP attributes, its local AS number is retrieved.

2. **Compute Advertised Networks**
   For each interface, the script determines whether the subnet mask corresponds to a routable network (e.g., /24, /28, /32). If valid, the network prefix is calculated and added to the set of BGP networks to be advertised.

3. **Discover Direct BGP Neighbors**
   The script scans all links to identify directly connected devices. If a connected neighbor also runs BGP:

   - The remote IP address is extracted.
   - The neighbor's AS number is retrieved.
   - The session type is classified as:
     - iBGP (same AS)
     - eBGP (different AS)
   - A neighbor entry is created and stored.

4. **Handle Bridge-Based iBGP Sessions**
   If the adjacent device is a bridge, the script inspects the bridge's other links. If another connected node belongs to the same AS, an indirect iBGP session is established via the bridge. This ensures full iBGP connectivity inside the AS even when intermediate Layer-2 devices are present.

2.2.4 Template Rendering

Once all data for the node is computed (interfaces, BGP neighbors, and advertised networks), the script renders the Jinja2 template:

- `device`: full node definition
- `interfaces`: interface list
- `neighbors`: computed BGP neighbor definitions
- `networks`: calculated network prefixes

The rendered output is written to a file named `<hostname>.conf` inside the `configs` directory.

2.2.5 Automation Benefits

This approach provides:

- **Consistency**: All configurations follow the same template logic.
- **Scalability**: Adding a new device only requires updating `data.yaml`.
- **Reduced Human Error**: BGP neighbors and advertised networks are computed automatically.
- **Separation of Concerns**: Topology definition, logic, and configuration syntax are cleanly separated.

In summary, the script transforms a high-level topology model into fully generated, device-specific FRR configuration files through structured parsing, logical inference, and template rendering.

2.3 Output

Running the script generates individual `.conf` files for each router (e.g., `pe1.conf`, `gw1.conf`). These files are syntactically correct for FRR and automatically handle complex BGP requirements such as:

- Establishment of eBGP sessions
- Establishment of iBGP sessions within the Core (AS 65020).
- Correct propagation of customer prefixes throughout the network (announcing networks).

Chapter 3

Task 2: Network emulation with containerlab

The goal of this task was to create a lightweight, reproducible, and highly portable network simulation environment.

We used Containerlab, an open-source tool that orchestrates container-based networking labs. Containerlab parses a topology definition file and interacts with the Docker daemon to spawn containers and wire them together using virtual ethernet pairs (veth).

3.1 Components

- **Routing Plane**: We used the FRRouting (FRR) container image (frr/frr:v10.4.1) to simulate industry-standard routing capabilities.
- **Hosts**: We used lightweight Alpine Linux containers to simulate Customers (N1, N2) and Internet destinations (R1, R2, R3 loopback).
- **Layer 2 Switching**: A standard Linux Bridge was used to simulate the broadcast domain for the Core Network, interconnecting multiple routers (PE1, PE2, GW1, GW2) on a shared segment. This could be extended in the future using a more complex virtual switching capability like OpenSwitch.

3.2 Automation

The containerlab yaml file is generated through the `build_topology.py` script. This script takes as input the `data.yaml` file and generates the corresponding containerlab yaml file using the `topology.j2` template file.

```
name: project

topology:
    nodes:
      {% for node in nodes %}
      {{ node.name }}:
        {% if node.kind %}
```

```
        kind: {{ node.kind }}
        {% else %}
        kind: linux
        {% endif %}
        {% if node.kind != 'bridge' %}
        network-mode: none
        {% if node.role == 'host' %}
        image: alpine:latest
        env:
          MY_INTF: {{ node.interfaces[0].name }}
          MY_IP: {{ node.ipv4_address }}/24
          MY_GW: {{ node.gateway_ip }}
        binds:
        - ../configs/linux-host-cfg.sh:/linux-cfg.sh
        exec:
          - echo "script on {{ node.name }}"
          - sh /linux-cfg.sh
        {% else %}
        image: quay.io/frrouting/frr:10.4.1
        binds:
          - ../configs/{{ node.name }}.conf:/etc/frr/frr.conf
          - ../configs/frr/daemons:/etc/frr/daemons
          - ../configs/frr/vtysh.conf:/etc/frr/vtysh.conf
        {% endif %}
        {% endif %}
      {% endfor %}

    links:
      {% for link in links %}
          - endpoints: ["{{ link.a }}:{{ link.a_port }}", "{{ link.b }}:{{ link.b_port }}"]
      {% endfor %}

      {% for node in nodes %}
        {% if node.role != 'host' %}
          - type: dummy
            endpoint:
              node: {{ node.name }}
              interface: lo0
        {% endif %}
      {% endfor %}
```

A script `boostrap.sh` is used to deploy the containerlab infrastructure in a safe way, creating the bridge device calling `bridge.sh`, builds Docker image for the manager node.

3.2.1 Data Validation Phase

Before producing the output file, the function `validate_data(data)` performs several integrity checks on the topology model.

1. Duplicate IP Address Detection

The script scans all nodes and their interfaces to ensure that no IPv4 address is reused within the network.

For each interface:

- The IPv4 address is extracted.
- A dictionary (`seen_ips`) tracks already assigned IPs.
- If an IP is found more than once, a critical error is generated.

This prevents address conflicts that would otherwise lead to undefined routing behavior.

2. IP Address Format Validation

Each interface IP and subnet mask are combined and validated using Python's `ipaddress.ip_interface()` object.

If the IP format is invalid (e.g., malformed address or incorrect mask), the script records an error and stops execution.

3. Subnet Consistency on Links

For each link defined in the topology:

- The script retrieves the two endpoints (node:interface).
- If both interfaces have an IP configured, their network prefixes are compared.

Two checks are performed:

1. **Subnet Matching**: The two interfaces must belong to the same network. If their computed networks differ, a `SUBNET MISMATCH` error is raised.
2. **Mask Size Validation**: The subnet must allow at least two usable IP addresses. If the network contains fewer than two addresses (e.g., /32), it is considered invalid for a point-to-point link.

4. Error Handling

If any validation error is detected:

- All errors are printed.
- The script terminates with `sys.exit(1)`.

This ensures that topology generation is aborted if the input model is inconsistent.

3.2.2 Template-Based Topology Generation

If validation succeeds, the script proceeds with the generation phase:

- The Jinja2 engine loads the template `topology.j2`.
- The nodes and links defined in `data.yaml` are passed as template variables.
- The rendered output is written to `network.clab.yml`.

The generated file defines the container-based lab topology, typically used by Containerlab to instantiate the virtual network.

Chapter 4

Task 3: Traffic engineering

The core objective of this task is to implement an intelligent control plane node within the AS65020 network (the *Manager*) capable of dynamically optimizing traffic flows based on real-time demands. The system periodically receives a traffic prediction matrix representing the bandwidth requirements for upstream and downstream traffic and automatically adjusts BGP attributes to balance the load across the network's entry and exit points.

4.1 System Architecture and Traffic Matrix Generation

In a production environment, traffic matrices are typically constructed by collecting flow statistics (e.g., NetFlow or sFlow logs) from all edge routers, which are then aggregated by a central collector to form a comprehensive view of network demand.

For the purpose of this project, to streamline the emulation environment while maintaining realistic operational conditions, we simulate this monitoring phase through a dedicated module, `generate_traffic.py`. Instead of parsing raw router logs, this script generates a synthetic traffic matrix that serves as the input for our optimization algorithms.

4.1.1 Traffic Matrix Structure

The generated traffic matrix represents the bandwidth demand (in Mbps) between the Customer Edge (CE) routers and the three external destinations reachable via the Upstream Providers. To simulate a realistic traffic pattern where not every customer communicates with every possible destination on the Internet simultaneously, the generation algorithm applies a specific sparsity constraint:

- **Random Demand:** Most entries in the matrix are assigned a random bandwidth value between 1 and 100 Mbps.
- **Zero-Value Constraint:** To mimic selective traffic flows, the generator forces at least one destination per source row to have zero traffic. This creates an unbalanced matrix that effectively tests the robustness of the optimization logic.

For instance, a resulting global traffic matrix generated by our system for the three destinations might look like the following (T):

$$T = \begin{bmatrix} & Dest_1 & Dest_2 & Dest_3 \\ CE1 & 45 & 0 & 88 \\ CE2 & 0 & 33 & 67 \end{bmatrix}$$

This single data structure is then processed and **logically decomposed** into two distinct views. These sub-matrices serve as the specific inputs required by our two optimization scripts:

1. **CE-to-PE View ($T_{CE \to PE}$):** This logical view preserves the granularity of the original flows. It maps exactly how much traffic each Customer Edge sends towards specific destinations. This detailed breakdown is crucial for the *Ingress Optimizer*, which needs to make discrete decisions for each source-destination pair to determine which Provider Edge (PE1 or PE2) is best suited to handle the incoming packets to minimize load imbalance.

$$T_{CE \to PE} = \begin{bmatrix} & Dest_1 & Dest_2 & Dest_3 \\ CE1 & 45 & 0 & 88 \\ CE2 & 0 & 33 & 67 \end{bmatrix}$$

2. **PE-to-GW View ($T_{PE \to GW}$):** This logical view represents the **aggregated** traffic load at the Provider Edges after the ingress optimization has taken place. Once flows have been assigned to specific PEs, this matrix consolidates the volume of traffic that must be routed from each PE towards the Gateways. It serves as the input for the *Egress Optimizer* to determine the best exit Gateway (GW1 or GW2) for each destination, considering the capacity limits of the upstream links.

$$T_{PE \to GW} = \begin{bmatrix} & Dest_1 & Dest_2 & Dest_3 \\ PE1 & 45 & 0 & 88 \\ PE2 & 0 & 33 & 67 \end{bmatrix}$$

4.2 Optimization Strategy

To optimize the inbound and outbound traffic within the transit AS, we adopted a mathematical approach based on **Integer Linear Programming (ILP)**. While heuristic methods could provide approximate solutions, an ILP formulation guarantees finding the global optimum for load balancing.

The problem is divided into two sequential optimization models:

- **Ingress Optimization:** Focuses on minimizing the load difference between the two entry routers (`PE1` and `PE2`).
- **Egress Optimization:** Focuses on minimizing the maximum link saturation on the exit gateways (`GW1` and `GW2`), accounting for potentially asymmetric upstream capacities.

4.2.1 Handling Null Traffic and Default Routing

It is important to note that both optimization algorithms are designed to process only **active flows** (non-zero traffic). If the traffic matrix indicates a volume of 0 Mbps for a specific source-destination pair, the optimizer does not calculate a specific path or policy for that flow.

Consequently, no specific BGP attributes (MED or Local Preference) are enforced for these null traffic pairs. In the network configuration, this is handled implicitly via the default routing behavior. Since no specific Route Map entry matches these flows, they fall through to a generic **"permit all"** rule or simply follow the standard BGP path selection process. This ensures that even if a flow is not explicitly optimized (or if new traffic patterns emerge that were not in the prediction matrix), packets are never dropped but simply routed without specific traffic engineering metrics.

4.2.2 Actuation Mechanism: BGP Attributes

The calculated optimal decisions for active flows must be translated into network protocols to be effective. To achieve this, the system leverages specific BGP path attributes:

- **Ingress Control:** The results of the PE selection optimization are applied using the **Multi-Exit Discriminator (MED)**, influencing how external Customer Edge routers send traffic to AS65020.
- **Egress Control:** The results of the Gateway optimization are applied using **Local Preference**, instructing internal routers on which upstream link to prefer for outbound traffic.

This mapping turns the abstract mathematical solution into concrete routing policies.

The following sections detail the mathematical formulation and implementation of these two models.

4.3 Ingress Traffic Engineering: PE Selection Optimization

Within the network architecture, AS65020 (the Transit AS) receives traffic from customer networks (CE1 and CE2) through two primary entry points: `PE1` and `PE2`. The objective of this optimization process is to balance the computational and bandwidth load across these frontier routers, preventing any single point of entry from becoming a bottleneck. Mathematically, the goal is to minimize the load imbalance between the two Provider Edge (PE) routers.

4.3.1 Problem Definition

Let n represent the total number of traffic flows (source-destination pairs) identified within the traffic matrix. We define the following parameters:

- c_i: The traffic volume (in Mbps) of the i-th flow, where $i \in \{1, \ldots, n\}$.
- $x_i \in \{0, 1\}$: A binary decision variable where $x_i = 0$ indicates the flow is assigned to `PE1`, and $x_i = 1$ indicates assignment to `PE2`.

The aggregate loads S_1 and S_2 for the two ingress routers are defined as:

$$S_1 = \sum_{i=1}^{n} c_i(1 - x_i), \quad S_2 = \sum_{i=1}^{n} c_i x_i$$

4.3.2 Initial Formulation

The ideal objective is to minimize the absolute difference between the loads of the two routers. In its non-linear form, the problem is expressed as:

$$\min \quad |S_1 - S_2|$$

$$\text{subject to: } x_i \in \{0, 1\} \quad \forall i \in \{1, \ldots, n\}$$

While conceptually simple, the absolute value function is non-linear and non-differentiable, making it difficult to solve using standard optimization tools.

4.3.3 Mixed-Integer Linear Programming (MILP) Model

To solve the problem efficiently, we introduce an auxiliary continuous variable $t \in R_{\geq 0}$ representing the upper bound of the imbalance. The minimization of $|S_1 - S_2|$ is reformulated by linearizing the absolute value through a set of inequality constraints:

$$\min \quad t$$

$$\text{subject to:}$$

$$\begin{aligned} S_1 - S_2 &\leq t \\ S_2 - S_1 &\leq t \\ x_i &\in \{0, 1\} \quad \forall i \\ t &\geq 0 \end{aligned}$$

By expressing S_1 in terms of the total traffic volume $C_{tot} = \sum c_i$ (where $S_1 = C_{tot} - S_2$), the constraints are simplified to:

$$C_{tot} - 2\sum_{i=1}^{n} c_i x_i \leq t$$

$$2\sum_{i=1}^{n} c_i x_i - C_{tot} \leq t$$

Model Classification

This model is formally classified as a **Mixed-Integer Linear Programming (MILP)** problem:

- **Integer:** The variables x_i are binary. This restriction is necessary because the BGP protocol does not natively support splitting a single prefix across multiple ingress points; each flow must therefore be routed entirely through one PE.
- **Mixed:** The model is considered *mixed* because it combines discrete decision variables (x_i) with a continuous optimization variable (t).

4.3.4 Implementation and Resolution

The optimization problem is solved using the `scipy.optimize.milp` library. Once the optimal values for x_i are obtained, the `Manager` node translates these mathematical decisions into BGP routing policies. Specifically, it applies different **Multi-Exit Discriminator (MED)** values to influence inbound traffic and steer it toward the selected PE.

4.3.5 From Optimization to Configuration: BGP MED

After computing the optimal decision vector, the `Manager` acts as an orchestrator to implement these results on the FRR routers. For Ingress Traffic Engineering, the strategy relies on manipulating the **MED** attribute.

BGP uses the MED value to suggest to external neighbors (the Customer Edge routers) which entry point is preferred for a specific destination. Given equal conditions, the external router will select the path with the lowest MED value. The `Manager` implements this logic by iterating through each source-destination flow and assigning weights as follows:

- **Preferred Path:** The PE router selected by the optimizer is assigned a MED of `100`.
- **Backup Path:** The other PE router is assigned a MED of `200`.

This configuration ensures the calculated load balancing while providing intrinsic **fault-tolerance**. Should the preferred PE fail, the customer router will automatically switch to the backup path (MED 200), maintaining service continuity. The implementation of this logic in the `manager.py` script is shown below:

```python
# Iterating through the optimizer results
for i, src in enumerate(sources):
    for j, dst in enumerate(destinations):
        choice = int(ce_pe_dec_matrix[i][j]) # Selected PE (1 or 2)

        for pe_idx in [1, 2]:
            pe_name = f"pe{pe_idx}"
            # MED Assignment: 100 for preferred, 200 for backup
            med_val = 100 if pe_idx == choice else 200
```

```

            # Execute vtysh commands on the specific router
            set_med(pe_name, src, med_val, dst, seq_med)
```

Listing 4.1: Translating the optimization vector into BGP parameters

The `set_med` function interacts directly with the specific container's shell, injecting the necessary `route-map` configurations and updating BGP adjacencies in real-time to apply the new routing policies.

4.4 Egress Traffic Engineering: Gateway Selection Optimization

Once traffic has entered the network and has been processed by the Provider Edge (PE) routers, the next challenge is to manage its exit toward the Upstream Providers. While the first optimization module focuses on balancing absolute volumes at the ingress, the **Egress Traffic Engineering** module (`optimizer_PE_GW.py`) addresses a more complex constraint: **link capacity**.

In a real-world scenario, the links connecting `GW1` and `GW2` to their respective upstream providers may have different bandwidth limits. Simply balancing the volume of traffic (as done for the PEs) could lead to the saturation of a lower-capacity link while a higher-capacity one remains underutilized. Therefore, this second model is introduced to optimize the **percentage saturation** of the outbound links.

4.4.1 Problem Definition

For this stage, the input is the aggregated traffic matrix resulting from the PE selection. Let m be the number of aggregated flows from the PEs to the external destinations. We define:

- c_j: The aggregated traffic volume (in Mbps) for the j-th flow.
- cap_1, cap_2: The total bandwidth capacities (in Mbps) of the upstream links connected to `GW1` and `GW2`, respectively.
- $y_j \in \{0, 1\}$: A binary decision variable where $y_j = 0$ indicates the flow is assigned to `GW1`, and $y_j = 1$ indicates assignment to `GW2`.

The total load on each gateway is:

$$L_1 = \sum_{j=1}^{m} c_j(1 - y_j), \quad L_2 = \sum_{j=1}^{m} c_j y_j$$

4.4.2 Mathematical Model: Minimax Saturation

The objective is to minimize the maximum link utilization (saturation ratio) across the available gateways. This is a **Minimax** optimization problem. We introduce an auxiliary continuous variable t representing the maximum saturation ratio:

$$\min \quad t$$

$$\text{subject to:}$$

$$\frac{L_1}{cap_1} \leq t \implies \sum_{j=1}^{m} c_j(1 - y_j) \leq t \cdot cap_1$$

$$\frac{L_2}{cap_2} \leq t \implies \sum_{j=1}^{m} c_j y_j \leq t \cdot cap_2$$

$$y_j \in \{0, 1\} \quad \forall j, \quad t \geq 0$$

By minimizing t, the solver ensures that neither link is disproportionately congested relative to its capacity, effectively "flattening" the saturation levels across the network's egress points.

Model Classification

Similar to the ingress model, this is a **Mixed-Integer Linear Programming (MILP)** problem. The binary variables y_j represent the discrete choice of an exit gateway for a specific destination prefix, while the continuous variable t represents the saturation level.

4.4.3 From Optimization to Configuration: BGP Local Preference

To enforce the optimal egress paths calculated by the model, the `Manager` node manipulates the BGP **Local Preference (LP)** attribute on the PE routers.

Unlike MED, which is used to influence external ASes, Local Preference is an attribute communicated within the local AS (iBGP). It tells the internal routers which exit point is preferred: the path with the **highest** Local Preference is always chosen. The `Manager` applies the following logic:

- **Preferred Gateway:** The path toward the GW selected by the optimizer is assigned an LP of `200`.
- **Backup Gateway:** The alternative path is assigned an LP of `100`.

This ensures that all traffic for a specific destination prefix originating from or passing through a PE will prefer the mathematically optimal Gateway. The implementation in `manager.py` is as follows:

```
# Iterating through the PE-to-GW optimization results
for i, pe_name in enumerate(pes):
    for j, dst in enumerate(destinations):
        gw_choice = int(pe_gw_dec_matrix[i][j]) # Selected GW (1 or 2)

        if gw_choice == 0: continue

```

```
        for gw_idx in [1, 2]:
            gw_name = f"gw{gw_idx}"
            # Local Preference: 200 for preferred (higher is better), 100 for backup
            lp_val = 200 if gw_idx == gw_choice else 100

            # Apply configuration to the PE router
            set_local_pref(pe_name, gw_name, lp_val, dst, seq_lp)
```

Listing 4.2: Translating GW optimization into Local Preference

By combining **Ingress MED control** and **Egress Local Preference control**, the automation system achieves end-to-end traffic engineering, ensuring that flows are balanced upon entry and optimized based on capacity upon exit.

4.5 Automated Configuration Deployment

In the previous sections, we described the optimization logic and introduced the Python functions `set_med` and `set_local_pref` as the tools used to apply the calculated decisions. These functions are the operational core of the **Manager** node.

As defined in the Containerlab topology (`network.clab.yml`), the Manager is a specialized container connected to the management network (LAN). Its role is to act as the central orchestrator: it takes the mathematical results from the optimization algorithms and translates them into actual configuration commands sent to the network devices.

4.5.1 Interaction with the Emulated Control Plane

To modify the BGP configurations dynamically, the Manager needs a way to communicate with the routers (`PE`s and `GW`s). In a standard production network, a management server would typically connect to routers over the network (e.g., via SSH) to issue configuration commands.

In our emulated environment, we implemented a more direct approach leveraging the capabilities of Containerlab and Docker. The Manager container is deployed with the host's Docker socket mounted inside it (volume: `/var/run/docker.sock:/var/run/docker.sock`). This specific configuration grants the automation script running inside the Manager the ability to execute commands directly inside the other containers of the topology.

Therefore, when the functions `set_med` or `set_local_pref` are called, the script does not open a network connection. Instead, it constructs a command string and injects it into the target router using the `docker exec` mechanism. This allows the Manager to interact with the FRRouting (FRR) integrated shell (`vtysh`) as if a network administrator were typing commands directly on the router's console.

4.5.2 Applying BGP Attributes via Route Maps

The configuration applied by the Manager is specific and granular, avoiding disruptions to the rest of the network traffic. The script utilizes **Route Maps** and **Prefix**

Lists to target only the traffic flows identified by the traffic matrix.

- **Ingress Control:** To set the MED, the Manager injects a Route Map applied *outbound* towards the Customer Edge neighbors. This map matches the specific destination IP and sets the metric to 100 (for the preferred path) or 200 (for the backup path).
- **Egress Control:** To set the Local Preference, the Manager injects a Route Map applied *inbound* from the Gateway neighbors. It assigns a higher Local Preference value to the routes received from the optimal Gateway determined by the algorithm.

Finally, to ensure that these changes take effect immediately without dropping active connections, the script issues a `clear bgp ... soft` command. This forces the routers to re-process the routing updates and apply the new policies instantly, completing the automation loop.

Chapter 5

Deployment and Automation Pipelines

The entire lifecycle of the project, from dependency installation to network deployment and traffic optimization, is managed through a series of automated pipelines. These scripts are designed to minimize manual interaction with the shell and ensure a reproducible execution environment. The workflow is architecturally divided based on the execution context: the **Host Machine** (the user's physical computer) and the **Virtual Machine** (a Linux environment, typically hosted on the user's machine, where the network emulation takes place).

5.1 Host Machine Setup: `install.py`

The entry point for the local environment setup is the `install.py` pipeline. This script is intended to be executed on the host computer. Its primary function is to parse the `requirements.txt` file and automatically install all the Python libraries required to run the subsequent orchestration scripts. This ensures that the local environment has all the necessary dependencies (such as `paramiko`, `scp`, and `jinja2`) installed and ready before attempting to interact with the virtual infrastructure.

5.2 Orchestration and Deployment: `orchestrator.py`

Once the local dependencies are satisfied, the deployment process is triggered by the `orchestrator.py` script. This pipeline acts as the bridge between the development environment and the runtime environment.

5.2.1 Artifact Generation

Before establishing a connection with the VM, the orchestrator locally executes two crucial generation scripts to prepare the environment:

1. `build_topology.py`**:** This script parses the `data.yaml` file, which serves as the single source of truth for the network design. It translates the abstract

definition of nodes and links into a concrete `network.clab.yml` file compatible with Containerlab. It defines the specific Docker images to be used for each node type (e.g., standard FRR images for routers, and the custom Alpine image for the Manager).

2. `build_configs.py`: This script leverages the Jinja2 templating engine to generate the configuration files for every router in the topology. It iterates through the `data.yaml` structure and creates a specific `frr.conf` file for each node (CE, PE, GW, Upstream). These configurations include the assignment of IP addresses to interfaces, the definition of BGP neighbors, and the setup of the initial routing policies.

5.2.2 Synchronization with the VM

The project is designed to run on a Linux Virtual Machine that has *Containerlab* and *Docker* installed. The orchestrator reads the connection credentials from a `.env` file and establishes an SSH connection with the VM.

Crucially, the script employs the SCP protocol to perform a **comprehensive synchronization** of the project artifacts. It mirrors the essential local directory structure onto the remote VM, ensuring that every file and dependency required for the network emulation is present. The transfer includes:

- The `configs/` directory, which contains the specific `.conf` files for every node in the topology (CEs, PEs, GWs, Upstreams) generated in the previous step.
- The `automation/` directory, housing the core Python logic including `manager.py`, `handle_traffic.py`, and the optimization modules.
- The infrastructure lifecycle scripts (`bootstrap.sh`, `teardown.sh`) and the `Dockerfile` required to build the Manager node.
- The `tests/` directory, containing the validation scripts (`pings.py`, `traceroute.py`) needed for the verification phase.

This ensures that the runtime environment inside the VM is perfectly synchronized with the local development version.

5.3 Infrastructure Bootstrapping: `bootstrap.sh`

Once the files are present on the VM, the execution control passes to the `bootstrap.sh` pipeline. This shell script is responsible for transforming the static files into a running network emulation.

5.3.1 Connectivity and Environment Checks

The script first verifies the internet connectivity of the VM. This step is critical because the deployment requires pulling Docker images from external registries. If

the VM is offline, the script automatically attempts to request an IP address via a DHCP client to restore connectivity, ensuring the deployment does not fail due to network issues.

5.3.2 Building the Manager Node

A key step in this pipeline is the preparation of the **Manager Node**. Unlike standard router containers that use a generic FRR image, the Manager requires a specialized environment to perform Traffic Engineering. The bootstrap script invokes `docker build` to create a custom image (`acn-manager`) based on Alpine Linux. During this build process, the necessary scientific libraries, specifically `numpy` and `scipy` for the optimization algorithms, are pre-installed. This process ensures that when the Manager container starts, it is immediately ready to execute the complex mathematical models without requiring further setup or downloads.

5.3.3 Topology Instantiation

Finally, the script invokes `containerlab deploy`. This command reads the topology file and spins up the containers for all network nodes (Routers, Switches, and the Manager), linking them according to the defined virtual ethernet pairs and bridges.

5.3.4 Environment Cleanup: `teardown.sh`

Included in the suite of scripts transferred to the VM is `teardown.sh`. This pipeline performs the inverse operations of the bootstrap: it destroys the Containerlab topology, stops the running containers, and removes the virtual bridges. While not strictly necessary for a single run, this script is essential during the development and testing phase, as it ensures the VM can be returned to a clean state before a new deployment.

5.4 Traffic Engineering Execution: `manager.py`

With the infrastructure active, the final step is to execute the core logic of the project. This requires accessing the shell of the Manager container:

```
docker exec -it clab-project-manager sh
```

From within this container, the `manager.py` pipeline is launched. This script acts as the central intelligence of the project and automates the entire control loop through several distinct phases:

1. **Traffic Generation:** It calls the generation module to produce the synthetic traffic matrix, applying the sparsity constraints (zero-values) to simulate realistic flows.

2. **Data Ingestion:** It parses the topology and the traffic matrix to construct the internal data structures required for optimization.

3. **Optimization:** It executes the two mathematical models sequentially. First, the **Ingress Optimizer** balances the load between PEs; next, the **Egress Optimizer** distributes the aggregated traffic across Gateways based on link capacity.

4. **Actuation:** It connects to the routers via the Docker socket. Using the `handle_traffic` module, it translates the optimal decisions into FRR commands and injects them via `vtysh`, applying Route Maps for MED and Local Preference.

5. **Reporting:** Finally, it exports the results to a JSON file and prints a detailed report to the console.

Upon completion of the `manager.py` execution, the network is fully configured and optimized. At this stage, the environment is ready for validation. The project includes specific testing scripts (`pings.py`, `te.py`, `traceroute.py`) to verify the reachability and the correctness of the routing paths, which will be detailed in the following chapter.

5.5 Manager Execution Output

The console output of `manager.py` provides a real-time view of the automation logic. Figures 5.1 and 5.2 illustrate the complete control loop.

Figure 5.1 details the traffic generation and the **CE → PE Optimization**. The sparsity of the traffic matrix is reflected in the decision vector (e.g., `[0 1 2 ...]`). Here, `1` and `2` denote the selected PE, while `0` indicates a flow with null volume. Crucially, the optimizer ignores these zero-traffic flows. Consequently, no specific BGP attributes (MED) are enforced, allowing these packets to follow standard default routing without explicit engineering.

Figure 5.2 shows the **Aggregated Load** at the PEs and the subsequent **Egress Optimization**. The algorithm applies Minimax Saturation to select the optimal Gateway based on link capacity; again, zeros appear where no aggregated traffic exists. Finally, the script configures BGP Local Preference (200 for preferred paths, 100 for backup) and exports the routing decisions to a JSON file, ready for the verification phase described in the next chapter.

```
student@student1-2025-amd:~/ACN_bgp-automation$ docker exec -it clab-project-manager sh
~ # python3 automation/manager.py

========== STEP 1: Traffic Generation ==========
--- Topology Discovery ---
Source Routers (CE):      ['ce1', 'ce2']
PE Routers:               ['pe1', 'pe2']
Destination Routers:      ['r1', 'r2', 'r3']

--- Generating Traffic Matrix (Source -> Destination) ---

-----------------------------------------------------
Source / Destination |       r1 |       r2 |       r3
-----------------------------------------------------
ce1                  |        0 |       16 |       11 |
ce2                  |       39 |       40 |        0 |
-----------------------------------------------------

TRAFFIC POLICY: 1 zero per row, others 1-100 Mbps.

[OK] Traffic matrix saved to: /root/automation/traffic_matrix.json

========== STEP 2: Data Acquisition ==========

========== STEP 3: Optimization CE -> PE ==========
[DEBUG] 1D CE-PE Vector:
[0 1 2 1 2 0]

CE -> DESTINATION            | ASSIGNED PE
ce1          -> r2           | PE 1
ce1          -> r3           | PE 2
ce2          -> r1           | PE 1
ce2          -> r2           | PE 2

========== STEP 4: BGP MED Configuration (PE Selection) ==========
CE (Flow Source)     | DESTINATION     | Node (PE)           | MED
ce1                  | r2              | pe1                 | 100
ce1                  | r2              | pe2                 | 200
-------------------------------------------------------------------
ce1                  | r3              | pe1                 | 200
ce1                  | r3              | pe2                 | 100
-------------------------------------------------------------------
ce2                  | r1              | pe1                 | 100
ce2                  | r1              | pe2                 | 200
-------------------------------------------------------------------
ce2                  | r2              | pe1                 | 200
ce2                  | r2              | pe2                 | 100
-------------------------------------------------------------------
```

Figure 5.1: Manager Execution: Traffic Generation and Inbound Optimization.

```
========== STEP 5: Aggregated Load Matrix (PE -> DEST) ==========
PE / DEST     |        r1 |        r2 |        r3
---------------------------------------------
pe1           |        39 |        16 |         0
pe2           |         0 |        40 |        11

========== STEP 6: PE -> GW Optimization (Minimax Saturation) ==========
[INFO] Capacities loaded from YAML: GW1=1000.0, GW2=1000.0
[DEBUG] 1D PE-GW Vector:
[1 1 0 0 2 2]

PE -> DESTINATION           | ASSIGNED GW
pe1         -> r1           | GW 1
pe1         -> r2           | GW 1
pe2         -> r2           | GW 2
pe2         -> r3           | GW 2

========== STEP 7: BGP Local Preference Configuration (GW Selection) ===
PE (Flow Source)      | DESTINATION     | Node (GW)           | LOCAL PREF
pe1                   | r1              | gw1                 | 200
pe1                   | r1              | gw2                 | 100
-----------------------------------------------------------------
pe1                   | r2              | gw1                 | 200
pe1                   | r2              | gw2                 | 100
-----------------------------------------------------------------
pe2                   | r2              | gw1                 | 100
pe2                   | r2              | gw2                 | 200
-----------------------------------------------------------------
pe2                   | r3              | gw1                 | 100
pe2                   | r3              | gw2                 | 200
-----------------------------------------------------------------

========== STEP 8: End-to-End Paths Export ==========
[OK] Final JSON saved: /root/automation/final_routing_paths.json
```

Figure 5.2: Manager Execution: Load Aggregation and Outbound Optimization.

Chapter 6

Testing and Results

To ensure the reliability of the emulated network and the correctness of the optimization algorithms, a comprehensive testing framework was developed. The validation process is divided into two sequential stages:

1. **Data Plane Readiness:** Verifies basic IP connectivity, routing table consistency, and default gateway configuration on end hosts.
2. **Traffic Engineering Verification:** Validates that the BGP policies injected by the Manager effectively influence packet forwarding paths as calculated by the optimization models.

All tests are automated using Python scripts that interact with the containerized nodes via the Docker engine.

6.1 Data Plane Connectivity and Routing

Before applying complex Traffic Engineering policies, it is essential to certify the underlying network infrastructure. Two scripts were implemented for this purpose: `pings.py` for bidirectional reachability and `traceroute.py` for host routing verification.

6.1.1 Bidirectional Reachability Analysis

The script `tests/pings.py` performs end-to-end ICMP tests to verify that the core network correctly routes traffic between internal hosts and external routers. The validation logic covers two distinct scenarios:

- **Host-to-Router (Outbound):** Verifies that client nodes (e.g., `n1`, `n2`) can reach the external infrastructure (e.g., `r1`, `r2`). This confirms that the internal routing protocols (OSPF/BGP) and NAT configurations (if present) are functional.
- **Router-to-Host (Inbound/Return):** Verifies that external routers can reach the internal hosts. Crucially, this test forces the source IP address using the `-I` flag (e.g., `ping -I 8.8.8.8 <host_ip>`). This distinction is fundamental for the validity of the test:

- **Without `-I`:** The router would use the IP address of its egress physical interface as the source. Since this interface is directly connected to the upstream provider, the return path is trivial and does not verify the global routing table.
- **With `-I`:** The router forces the packet to originate from its Loopback address (e.g., `8.8.8.8`). This address is logical and does not exist "physically" on the transit link; it resides only within the router's configuration. Consequently, for the ping to succeed, the ICMP Echo Reply must find its way back. This proves that the **entire network chain** (UP, GW, PE, CE) has correctly learned via BGP that the route to reach `8.8.8.8` passes through the specific Gateway, validating the control plane end-to-end.

The script dynamically parses the `data.yaml` topology file to identify source nodes and target IP addresses, executing commands via:

```
docker exec <container> ping -c 3 -W 2 <destination_ip>
```

6.1.2 Host Routing Verification

The script `tests/traceroute.py` focuses on the correct configuration of the end-hosts. Since `n1` and `n2` are Linux containers simulating client machines, they must have a correct default gateway pointing to their respective Customer Edge (CE) routers.

The verification procedure iterates through the hosts defined in the topology:

1. Retrieves the expected `gateway_ip` from the configuration file.
2. Executes a traceroute to a mock external destination (e.g., `8.8.8.8`).
3. Analyzes the first hop of the output.

If the IP address of the first hop matches the configured gateway, the test is marked as **PASS**; otherwise, it indicates a misconfiguration in the host's routing table.

6.2 Traffic Engineering Validation

The core validation of the project is performed by the `tests/te.py` script. Its objective is to verify that the "Optimization → Manager → Network" pipeline works correctly, meaning that the mathematical decisions are effectively translated into forwarding behaviors in the data plane.

6.2.1 Methodology

Unlike simple connectivity tests, this script validates the *exact path* taken by packets. It relies on two inputs:

- `data.yaml`: The physical topology definition.

- `final_routing_paths.json`: The output of the Manager's optimization phase, listing the expected Provider Edge (`PE`) and Gateway (`GW`) for each traffic flow.

For each source-destination pair optimized by the Manager, the script performs a **traceroute** from the source CE and compares the actual network hops against the expected path.

6.2.2 Link-Level IP Resolution

A critical challenge in automated traceroute analysis is that the output displays the IP addresses of the *ingress interfaces* of the routers, not their loopback or system IDs. To address this, the script implements a logic function `get_link_ip(topology, node_a, node_b)` that:

1. Identifies the physical link connecting `node_a` and `node_b` in the YAML topology.
2. Determines the specific interface port on `node_b` facing `node_a`.
3. Resolves the IPv4 address assigned to that specific interface.

This allows the script to predict exactly which IP address should appear in the traceroute output if the traffic passes through the correct link.

6.2.3 Validation Logic

The verification process follows these steps for every active flow:

1. **Path Extraction:** Reads the expected `PE` and `GW` from the JSON file.
2. **Execution:** Runs `docker exec ... traceroute -n -w 1 <dest_ip>` on the source container. The `-n` flag prevents DNS resolution to speed up execution.
3. **Hop Verification:**
   - Checks if the resolved IP of the link `Source` → `PE` appears in the output.
   - Checks if the resolved IP of the link `PE` → `GW` appears in the output.

6.2.4 Results Interpretation

The script generates a summary table on the console. A flow is considered:

- **[OK]**: If both the expected PE and GW hops are detected in the traceroute. This confirms that BGP `MED` (for PE selection) and `Local Preference` (for GW selection) were correctly applied and obeyed by the routers.
- **[FAIL]**: If one or both hops are missing or different from the expectation. This triggers a debug output showing the expected vs. actual IPs to facilitate troubleshooting.

This automated approach provides definitive proof that the Traffic Engineering system is not just a theoretical model but actively controls the network traffic.

Chapter 7

Future Work and Scalability Analysis

While the current implementation successfully demonstrates the core concepts of automated Traffic Engineering within AS65020, there are several areas where the system could be expanded and refined to meet the demands of a larger, production-grade network.

7.1 Topology Generalization and Testing

The software architecture was designed with modularity in mind. The scripts rely on the `data.yaml` file as the single source of truth for the network topology, and the optimization logic iterates over dynamic lists of routers (Sources, PEs, Gateways) rather than hardcoded variables. Theoretically, this allows the system to support an arbitrary number of Customer Edges, Provider Edges, or Upstream Providers.

However, the current validation has been strictly focused on the reference topology (2 PEs, 2 GWs). Consequently, while the mathematical models are generic, the full automation pipeline has not been rigorously tested with a larger number of nodes (e.g., adding a third PE or a third Upstream Provider). We anticipate that introducing such changes might require minor adjustments to the integration logic. We are confident, however, that these would be minimal code refactoring tasks rather than structural changes.

7.2 Algorithmic Scalability

A more significant challenge lies in the computational complexity of the chosen optimization strategy. Our solution employs **Mixed-Integer Linear Programming (MILP)** to guarantee mathematically optimal load balancing.

While efficient for the scale of our project, MILP is known to be an **NP-hard** problem. The complexity of the Branch-and-Bound algorithms used by the solver grows exponentially with the number of integer decision variables. In a real-world scenario with thousands of traffic flows and dozens of edge routers, finding the global optimum might require computational time that exceeds the strict timing requirements of real-time traffic engineering.

To address this scalability bottleneck in a future iteration, the exact MILP approach could be replaced or augmented with **Heuristic algorithms** (such as Greedy approaches). While heuristics do not guarantee the theoretically perfect solution, they offer polynomial time complexity, ensuring that the system remains responsive and scalable even as the network size increases significantly.

Chapter 8

AI Usage Disclaimer

In the spirit of transparency and academic integrity, we declare that the conception, architectural design, and original implementation of this project are entirely our own work. The mathematical models for traffic engineering, the network topology, and the core logic of the automation scripts were developed independently.

To enhance the quality of the final deliverable and ensure better accessibility for future readers, we utilized Large Language Models from the **Google Gemini** family. These tools served exclusively as support instruments to refine the presentation of our work, improve code readability, and streamline the documentation process.

Specifically, AI assistance was employed for the following tasks:

8.1 Code Readability and Presentation

Given the complexity of the project, which involves numerous interacting scripts and configuration files, we used AI to improve the organization and clarity of the codebase.

- **File Headers and Docstrings:** To help readers quickly understand the purpose of each module, we used the AI to generate comprehensive descriptive headers at the beginning of every script. This provides an immediate overview of what each file (e.g., `manager.py`, `optimizer_*.py`) is responsible for.
- **Refactoring for Clarity:** While the logic remained ours, we leveraged AI suggestions to organize the code into cleaner functions, making the flow of execution easier to follow.
- **Console Output Enhancement:** We utilized the AI to implement a cleaner and color-coded logging system for the terminal. This choice was made to make the real-time monitoring of the network status more intuitive and visually organized during the system execution.

8.2 Documentation Support

Finally, Google Gemini models were used to review the text of this report, checking for grammatical correctness and improving overall flow and clarity.